

DCCA-GS: Decoder-Reproducible Content-Adaptive Quantization and Coverage-Aware Fusion Pruning for Anchor-Based 3D Gaussian Splatting

Chen Tong

Department of ..., University of ...

Email: todo@example.edu

Abstract—Anchor-based codecs such as HAC++ compress 3D Gaussian Splatting (3DGS) scenes by more than an order of magnitude, yet two decisions inside them remain crude. First, the quantization step of each anchor is predicted from hash context alone: the allocation ignores how complex the local content is and how much it matters to the rendered image, and carrying per-anchor step decisions in the bitstream would violate the zero-side-information contract that makes anchor codecs practical. Second, anchor pruning selects survivors with static, per-anchor scores, which is blind to spatial coverage: under a tight budget the score keeps high-sensitivity detail anchors and deletes the few anchors that cover large background regions, and reconstruction collapses. We present DCCA-GS, which addresses both problems with two decoder-reproducible mechanisms. (1) A *complexity multiplier* rescales the quantization step of every anchor by a factor computed from four structural statistics that the decoder can recompute exactly; during training the same small network is supervised by render-sensitivity statistics, so fine steps flow to anchors that matter to the image—without a single bit of side information. (2) *coverage-aware fusion pruning* augments the ADMM sparsification of GaussianSpa with a coverage-dominant score that fuses per-anchor coverage with render sensitivity, and with a hard projection that keeps at least one anchor per occupied coarse cell. On three large-scale UAV scenes, DCCA-GS needs on average 58% of the HAC++ model size at matched λ while remaining within 0.27–0.61 dB PSNR, ranks first among 18 compared methods on a per-scene rank aggregation, and its coverage guards eliminate the failure mode behind a 1.56 dB low-rate collapse at a quality cost within measurement noise.

I. INTRODUCTION

3D Gaussian Splatting (3DGS) [1] represents a scene with explicit Gaussian primitives and renders novel views in real time, but high-quality scenes require millions of primitives whose attributes dominate storage. Anchor-based codecs reduce this cost by organizing Gaussians around a compact set of anchors: Scaffold-GS [2] derives neural Gaussians from anchor features, HAC [3] and HAC++ [4] add hash-grid contexts and conditional entropy models, and ContextGS [5] models anchor-level context. These systems push the rate-distortion frontier far beyond vanilla 3DGS, and we build directly on the HAC++ coding pipeline. Two mechanisms inside this pipeline, however, allocate the remaining bits crudely, and both are the subject of this paper.

A. Bit allocation is content-blind

HAC++ predicts a *base* quantization step per anchor from hash-grid context (adaptive quantization module, AQM). The base step, however, is modulated neither by the complexity of the local content nor by the importance of the anchor to the rendered image: a flat wall and a cluttered facade that share the same hash context receive the same step. Making the step content-adaptive seems to require either an explicit per-anchor step array—side information that changes the coding contract and costs bits—or latents whose rate must itself be modeled. Our setting forbids both: we require *zero side information*, i.e., every signal that influences quantization or rate estimation must be recomputable by the decoder from the decoded bitstream alone.

The way out is to separate *what the step depends on* from *what guides it*. Structural statistics—local anchor density, predicted scaling anisotropy, predicted offset energy, and active mask ratio—are functions of quantities the decoder already has, so a small network over these statistics is *decoder-reproducible* by construction. What this network should *predict* is not directly observable at test time, but during training we can measure how much each anchor matters: the gradient of the rendering loss with respect to its attributes. We therefore train the shared network to regress a render-sensitivity target accumulated as an exponential moving average. The supervision exists only in the training objective; the bitstream and the decode path are untouched, and the encoder-decoder round trip remains bit-exact.

B. Pruning is coverage-blind

The coarsest rate lever in an anchor codec is the number of surviving anchors. Training-side sparsification [6] imposes a hard budget during optimization via ADMM, so survivors re-adapt to the final budget—strictly better than pruning after training. Yet the selection step itself is still a greedy top- k over *static*, *per-anchor* scores. Such scores cannot express the statement “some anchor must remain in this region”: under a tight budget, a score dominated by render sensitivity retains cluttered detail anchors and removes the sparse anchors that cover large background areas. The result is a *coverage hole*:

[TODO: overview figure: left = anchor codec pipeline with the complexity multiplier hooked into AQM; right = SPA projection step with the coverage constraint and the fused score]

Fig. 1. DCCA-GS overview. Both innovations are decoder-reproducible: the complexity multiplier lives in the quantization path, the coverage-aware fusion pruning lives in the training-side projection. Neither writes anything extra to the bitstream.

in our experiments, fusing sensitivity into the pruning score without any coverage term collapses low-rate reconstruction by 1.56 dB on scene 1-78 while leaving the anchor count almost unchanged—the pruner did not prune too much, it pruned the wrong anchors.

We fix this structurally rather than by re-tuning weights. The fused pruning score is made *coverage-dominant* (sensitivity is capped at a 0.3 share), and the ADMM hard projection is extended with a coverage constraint: every occupied coarse cell of the scene keeps at least one anchor. The constraint is a one-line change to the projection step, costs nothing at decode time, and eliminates the collapse failure mode. A pre-registered re-validation of the repaired score improves the tightest-budget point and holds size parity, but does not clear our improvement gates; we therefore report coverage-aware fusion pruning as a robustness mechanism rather than a rate-distortion gain, and analyze the signal defects behind this boundary in Section IV-D.

C. Contributions

- **Decoder-reproducible content-adaptive quantization.** A complexity multiplier m_i rescales the AQM step of anchor i from four structural statistics; a single shared $8 \rightarrow 32 \rightarrow 3$ network produces it, is supervised in training by render-sensitivity gradients, and is recomputed bit-exactly by the decoder. The bitstream carries no step array and no side information. Removing the multiplier costs 0.41 dB at $\lambda=0.002$ on 1-78.
- **Coverage-aware fusion pruning.** On top of the ADMM budget sparsification of GaussianSpa [6], which we adopt as-is as the base mechanism, we add (i) depth re-initialization at growth stop, which inserts surface candidates that must compete under the *existing* budget, and (ii) a sensitivity-aware pruning strategy: a coverage-dominant fusion score plus an ADMM coverage constraint. The strategy removes the failure mode behind the measured 1.56 dB low-rate collapse at a quality cost within measurement noise; Section IV-D quantifies this boundary and dissects the signal defects that limit the rate-distortion gains of importance-guided selection.
- We validate both mechanisms on three large-scale UAV scenes under a uniform protocol (30k iterations, seed 42, fp32 sizes, bit-exact decode), and document the negative results that shaped the design.

II. RELATED WORK

A. Compression of 3D Gaussian Splatting

Compact-3DGS methods reduce the primitive count and the per-primitive cost: LightGaussian [7] distills and prunes by global significance, Compact3DGS [8] learns a compact mask, Reduced3DGS [9] sensitively quantizes attributes, MesonGS [10] transforms attributes post-training, and PCGS [11] refines masks and quantization progressively. Anchor-based codecs organize primitives around shared anchors: Scaffold-GS [2] introduces view-adaptive anchor features, HAC [3] couples anchors with hash-grid contexts, HAC++ [4] adds intra-anchor context with a Gaussian mixture likelihood, and ContextGS [5] models anchor-level context autoregressively. DCCA-GS is implemented on the open-source gsplat rasterization library [12] and reproduces the HAC++ coding pipeline within it; all HAC++ reference numbers used for comparison are produced by the official HAC++ implementation under an identical protocol. DCCA-GS is orthogonal to these probability-modeling improvements: it changes *which* step each anchor is quantized with and *which* anchors survive, while keeping the HAC++ bitstream contract.

B. Rate-Distortion Optimization and Adaptive Quantization

Learned image compression optimizes a differentiable rate estimate jointly with distortion [13], [14] and allocates precision through learned latents. RDO-Gaussian [15], CompGS [16], and CAT-3DGS [17] transfer this recipe to 3DGS. In all of them, per-element precision is carried by the latent itself and its cost by the entropy model. Our constraint is stricter: no signal beyond the decoded bitstream may influence quantization. The complexity multiplier therefore consumes only structural statistics and is guided, in training only, by render sensitivity—a mechanism reminiscent of perceptual JND (just-noticeable-difference) weighting in image codecs, but realized as a decoder-reproducible network rather than a side information channel.

C. Sparsification and Anchor Placement

HAC [3] and HAC++ [4] promote offset sparsity with mask losses folded into rate estimation. GaussianSpa [6] sparsifies during training by alternating a smooth sub-problem with a hard top- k projection—the ADMM structure we adopt as the budget mechanism. Placement has been studied by Mini-Splatting [18], which re-initializes Gaussians from depth back-projection and simplifies by contribution; we borrow only its depth-reinitialization idea, run it once at growth stop, and force the new candidates to compete under the existing ADMM budget instead of growing the model. What is missing in all of these selection rules is spatial coverage: scores are per-anchor and static, so no rule can express “this region must stay covered”. Our fusion pruning contributes exactly this missing piece, both softly (coverage-dominant score) and structurally (a hard projection constraint).

III. METHOD

A. Preliminaries and Coding Contract

We build on an anchor-based codec with N anchors and K offsets per anchor. Each anchor carries features $a_i \in \mathbb{R}^{D_a}$, scaling parameters, offsets, and a binary offset mask; each valid offset instantiates one neural Gaussian whose color and opacity are decoded by shared MLPs from a hash grid. The codec is trained with the Lagrangian objective

$$\min_{\theta} D(\theta) + \lambda R(\theta), \quad (1)$$

where θ collects all coded parameters and R is a differentiable rate estimate. Following HAC++, the base quantization step of anchor i for field f (feature, scaling, or offset) is predicted from hash context by the AQM, $Q_{i,f}^{\text{AQM}}$.

Zero-side-information contract. The decoder may use: (i) the decoded bitstream, (ii) the stored entropy-model and decoder weights, and (iii) any function of already-decoded quantities (hash grid, anchor geometry, masks, decoded MLP outputs). Anything else—per-anchor steps, importance maps, sensitivity maps—is forbidden. Both mechanisms below respect this contract by construction; the encoder–decoder round trip is verified bit-exactly for every reported run.

B. Complexity Multiplier with Render-Sensitivity Supervision

1) *Multiplier:* The final step used to quantize anchor i , field f , is

$$Q_{i,f} = Q_{i,f}^{\text{AQM}} (1 + \tanh q_{i,f}^{\text{AQM}}) (1 + \alpha \tanh m_{i,f}), \quad (2)$$

where α ramps from 0 to a configured scale during training so that quantization-aware training starts from the HAC++ behaviour. The multiplier is produced by a small shared network, $m_{i,f} = [\text{MLP}_c(x_i)]_f$ with $\text{MLP}_c : 8 \rightarrow 32 \rightarrow 3$, from the input vector x_i of four structural statistics: *local anchor density* ρ_i (occupancy of the anchor’s voxel neighbourhood), *predicted scaling anisotropy* (spread of the decoded log-scales), *predicted offset energy* (norm of the decoded offsets), and *active mask ratio* (fraction of surviving offsets). All four are functions of quantities the decoder possesses after decoding the grid and the anchor attributes, so MLP_c —whose weights are stored in the bitstream as part of the entropy model—is evaluated identically at both ends. This is the same property that makes AQM itself decoder-reproducible; we extend it from a context-derived *base* step to a content-adaptive *modulation* of that step. Intuitively, MLP_c lowers the step (finer quantization) where content is dense, anisotropic, and high-energy, and raises it where few, isotropic, low-energy anchors cover smooth regions.

2) *Render-sensitivity supervision:* Nothing in x_i says whether the anchor *matters*. During training we measure exactly that: retaining gradients through the quantization-aware forward pass, we accumulate per-anchor sensitivity

$$s_i = \text{EMA}_{\tau} \left(\left\| \partial \mathcal{L}_{\text{render}} / \partial u_i \right\|_1 \right), \quad (3)$$

where u_i collects the pre-quantization attributes generated by anchor i and τ is the EMA decay. A bounded, per-frame-normalized target is derived from s_i , and MLP_c is trained with an additional MSE regression term weighted by λ_{sens} . The target exists only in the training objective: at test time the decoder runs MLP_c on x_i alone. The network therefore learns to *predict* sensitivity from structure, and the prediction modulates the step—fine steps flow to anchors that matter to the rendered image, coarse steps to anchors that do not.

C. Coverage-Aware Fusion Pruning

1) *Base mechanism: ADMM budget sparsification:* We adopt the training-side sparsification of GaussianSpa [6] as-is. With a_i the anchor mask score, a binary state z_i , dual variable u_i , and budget κ , each projection step solves

$$z = \text{TopK}(a + u, \kappa), \quad u = \text{clamp}(u + a - z, \pm 1), \quad (4)$$

and anchors with $z_i = 0$ are pruned while the survivors continue to train under the budget. The budget is ramped down from the historical maximum anchor count during the growth phase. This mechanism decides *how many* anchors survive; our additions decide *which* ones.

2) *Depth re-initialization under the fixed budget:* Anchor growth stops early in training, and the anchors are clustered where initial SfM points happen to be. Following the depth-reinitialization idea of Mini-Splatting [18]—and *only* that idea—we render depth maps for V sampled training cameras at growth stop, back-project them to surface points, voxelize them, and append the highest-coverage candidates as new anchors. Crucially, the SPA budget κ is *not* increased: the new candidates must displace existing anchors through the projection in (4). This lets the anchor set migrate toward the true scene surface without growing the model.

3) *Sensitivity-aware fusion score:* The projection in (4) ranks anchors by the static mask score a_i . We replace the ranking by a fused importance

$$\hat{s}_i = (1 - w) c_i + w t_i + \beta a_i, \quad w \leq w_{\text{max}} = 0.3, \quad (5)$$

where c_i is the *coverage* of anchor i (its accumulated projected area in the rasterizer’s per-anchor bookkeeping, normalized), t_i is the *render sensitivity* of Section III-B (normalized), and β keeps the original mask score as a tiebreaker. The cap w_{max} is the load-bearing detail: an unconstrained w lets sensitivity dominate, and we measured the consequence—on 1-78 at $\lambda=0.004$, an $w=0.5$ fusion collapses decoded PSNR from 27.51 to 25.95 dB at an almost unchanged anchor count (507.8k vs. 506.0k): the pruner kept high-sensitivity detail anchors and deleted low-sensitivity anchors that covered large background regions, opening coverage holes. With $w \leq 0.3$, coverage is always the dominant term.

4) *ADMM coverage constraint:* The soft cap is complemented by a structural guarantee. Partition the scene into coarse cells of size δ (we use $\delta = 0.05$ in scene units); at each hard projection, every occupied cell must keep at least one anchor:

$$\sum_{i \in C} z_i \geq 1 \quad \forall C \text{ occupied}, \quad (6)$$

Algorithm 1 DCCA-GS training loop (per iteration)

- 1: render, compute $\mathcal{L} = \mathcal{L}_{\text{render}} + \lambda R + \lambda_{\text{sens}} \|\text{MLP}_c(x) - s\|^2$ {Eq. (3)}
- 2: update anchor growth; accumulate coverage c_i and sensitivity s_i
- 3: **if** ADMM projection step **then**
- 4: $\hat{s}_i = (1 - w)c_i + wt_i + \beta a_i$ {Eq. (5), $w \leq 0.3$ }
- 5: protect $\arg \max_{i \in C} \hat{s}_i$ per occupied cell C ; project $z = \text{TopK}(\hat{s} + u, \kappa)$ {Eq. (6)}
- 6: update u ; prune $z_i = 0$
- 7: **end if**
- 8: **if** growth stop T_r **then**
- 9: back-project sampled depths; append voxelized candidates under fixed κ
- 10: **end if**
- 11: quantize with $Q_{i,f}$ {Eq. (2); MLP_c shared, decoder-recomputed}

TABLE I
ZERO-SIDE-INFORMATION AUDIT.

Signal	Producer	In bitstream?
Base step Q^{AQM}	hash context (HAC++)	weights only
Complexity input x_i	decoded grid/anchors/masks	no
Multiplier $m_{i,f}$	shared MLP_c	weights only
Sensitivity target s_i	training gradients	training only
Coverage c_i	rasterizer bookkeep- ing	training only
Fusion score $\hat{s}_i$	training-side projec- tion	training only

implemented by protecting the best-scoring anchor of each occupied cell before the top- k of (4). The constraint costs nothing at decode time and is unconditional: whatever the fused scores say, no region of the scene can go unrepresented. In our measurements the constraint alone rarely changes the anchor count (e.g., 671,570 vs. 671,497 on 4-10); its value is that the failure mode of (5) is impossible by construction, which is what allowed the fusion to be enabled at the low-rate operating points at all.

D. What the Decoder Recomputes

Table I audits the coding contract. Both innovations consume only decoder-recomputable signals at test time; the sensitivity target and the coverage statistics are training-side only and are never encoded. The complete encode $\rightarrow$ decode $\rightarrow$ evaluate pipeline is exercised on every reported operating point, and a bit-exact round-trip diagnostic confirms that the decoded model is identical to the one the encoder measured.

IV. EXPERIMENTS

A. Setup

We evaluate on three large-scale UAV scenes (1-78, 2-06, 4-10; 1,200 images each, 150 validation views). The default protocol trains for 30k iterations (60k in the iteration study) with

feature dimension 32, 10 offsets, seed 42, data_factor=1, 1600-px width, and one validation pass out of every 8 views. All sizes are decode-required payload with fp32 decoder weights (HAC++ accounting), and every operating point is produced by the full encode $\rightarrow$ decode $\rightarrow$ evaluate pipeline with a bit-exact round-trip diagnostic. Comparison numbers come from two sources: (i) a shared evaluation table covering 18 methods on the same scenes — compiled by a third party; the per-method protocol is not re-verified, so we treat it as a breadth reference and flag it accordingly; and (ii) our own runs of the *official* HAC++ implementation under an identical protocol on scene 1-78, which we use as the rigorous reference. DCCA-GS-base denotes the complexity multiplier (I2+I6) on top of SPA and depth re-initialization; DCCA-GS-full additionally enables coverage-aware fusion pruning.

B. Main comparison

Table II reports one operating point per method (PSNR / size in MB). On 1-78, the DCCA-GS-base point dominates every entry of the shared table on both axes simultaneously; on 2-06 and 4-10 it is the smallest model within 0.42 and 0.06 dB of the best entry, respectively. Aggregating the three scenes with the rank score $\text{rank}(\text{PSNR})/6 + \text{rank}(\text{SSIM})/6 + \text{rank}(\text{LPIPS})/6 + \text{rank}(\text{Size})/2$ (rank 1 best, averaged over scenes), DCCA-GS scores 3.17 versus 3.56 for HAC++, the best of the 18 compared methods. Against the stricter, same-protocol official HAC++ runs on 1-78 (27.84 dB at 20.0 MB, 28.21 at 26.3, 28.68 at 40.6), DCCA-GS-base at 30k trades quality for size (-0.27 to -0.61 dB at 58% of the size at matched λ).

C. Training iterations

Doubling the budget from 30k to 60k improves DCCA-GS-base by +0.44 dB at $\lambda=0.004$ (27.95 dB at 12.5 MB), +0.62 dB at $\lambda=0.002$ (28.37 at 16.7), and +0.23 dB at $\lambda=0.0005$ (28.18 at 25.8). At 60k the $\lambda=0.004$ and $\lambda=0.002$ points dominate the 30k HAC++ references on both axes; we emphasize the iteration mismatch: HAC++ likewise improves substantially with 60k (their reported BD-rate gain is -32.9%), so this is a budget-matched trend, not a claim against HAC++ at 60k.

D. Analysis: importance-guided pruning under sparse coverage

With fusion pruning engaged (verified by per-projection engagement logging in $> 89\%$ of projections), a six-point scoreboard against DCCA-GS-base shows the mechanism at parity overall (-0.20 , $+0.10$, $+0.08$ dB at 30k for $\lambda=0.004/0.002/0.0005$; -0.01 , -0.23 , -0.12 at 60k), with a consistent pattern: gains at mild budgets, losses where pruning pressure is highest. Instrumentation explains the pattern. The coverage signal is computed from a fixed sample of 8 training views and max-normalized; the resulting per-anchor vector is extremely skewed (mean ≈ 0.000 : a handful of anchors hold the entire normalized mass, the silent majority is exactly zero). The additive score $0.7 \tilde{c} + 0.3 \tilde{s} + 0.25 (a + u)$ therefore pins

TABLE II
DECODED-VIEW PSNR (dB) / MODEL SIZE (MB) ON THREE LARGE-SCALE UAV SCENES, ONE OPERATING POINT PER METHOD. SHARED-TABLE ENTRIES ARE SINGLE POINTS COMPILED BY A THIRD-PARTY EVALUATION (PROTOCOL NOT RE-VERIFIED); DCCA-GS ROWS ARE DECODED BITSTREAMS UNDER THE UNIFORM 30K/SEED-42/FP32 PROTOCOL AT $\lambda=0.002$.

Method	1-78	2-06	4-10
HAC++ [4]	27.14 / 17.2	28.51 / 21.0	28.76 / 17.4
PC-GS	23.09 / 12.0	26.26 / 15.1	26.36 / 11.8
CAT	23.40 / 14.3	25.72 / 15.4	26.29 / 13.1
ContextGS [5]	24.90 / 29.3	27.20 / 26.8	27.39 / 22.6
HAC [3]	24.83 / 21.7	26.86 / 26.8	26.94 / 20.9
RDO-Gaussian [15]	23.91 / 11.2	25.14 / 14.7	24.68 / 6.4
Scaffold-GS [2]	24.63 / 286.1	27.21 / 250.2	27.21 / 202.0
Compact3D	24.27 / 21.5	25.68 / 33.6	25.53 / 19.9
gsplat [12] (uncompressed)	26.02 / 604	27.63 / 606	26.82 / 610
3DGS [1] (uncompressed)	26.06 / 604	27.76 / 606	27.02 / 610
TC-GS	23.41 / 17.7	24.84 / 15.7	25.99 / 18.6
Compact3DGS [8]	24.38 / 59.6	25.75 / 86.1	25.40 / 51.4
Mini-Splatting [18]	23.60 / 21.3	25.11 / 30.3	25.20 / 28.6
FC-GS	19.51 / 47.1	20.25 / 59.6	21.94 / 59.7
LightGaussian [7]	24.35 / 281.7	26.22 / 407.5	25.46 / 194.9
GaussianSpa [6]	24.37 / 106.5	25.13 / 107.2	25.50 / 99.6
Octree-GS	20.33 / 21.5	21.29 / 15.7	24.55 / 34.7
DCCA-GS-base (I2+I6)	27.75 / 15.0	28.09 / 16.4	28.70 / 15.7

its scale on a few anchors and demotes the calibrated ADMM score to a third-order term for the unseen majority—exactly where ranking noise is executed most often, i.e., at the tightest budget. Two structural guards remained effective throughout: the sensitivity cap ($w \leq 0.3$) and the per-cell coverage constraint prevent the catastrophic failure mode of an uncapped fusion (a 1.56 dB collapse at $\lambda=0.004$), bounding the damage to parity. Based on this analysis the score was revised to a rotating-view coverage EMA (1080 distinct view samples over training), percentile normalization, and a multiplicative modulation $(a+u)(1+\gamma\tilde{z})$ that leaves the calibrated ADMM ranking of zero-coverage anchors untouched. The revision thickens the coverage distribution as designed (coverage p_{50} rises from 0.000 to 0.176) and improves the tightest-budget point (+0.06 dB at $\lambda=0.004$, now size-neutral), but does not clear our pre-registered improvement gates (≥ 27.46 at $\lambda=0.004$, ≥ 27.80 at $\lambda=0.002$ on 1-78; the latter passed by 0.009 dB). We therefore position coverage-aware fusion pruning as a robustness mechanism: its guards eliminate the collapse failure mode at an RD cost that is within noise, rather than as a rate-distortion improvement.

E. Negative results

We document the directions that did not survive measurement. (i) A DINO semantic prior is predictable from decoder-recomputable features ($r = 0.60$ with rich context) but is the wrong target: semantic identity does not correspond to quantization importance, and the predictor gate initially passed on an input set richer than the one the supervised network actually receives. (ii) An opacity-weighted reconstruction loss (per-pixel importance from rendered alpha) degrades PSNR at two of three λ settings. (iii) MLP weight quantization saves only $\sim 0.5\%$ of the payload and its original implementation

mixed quantized-size accounting with a float32 model; we removed it rather than report the inconsistent number. (iv) A sensitivity side-information variant increases the payload with no quality gain and is disabled. (v) Hierarchical context adds size for negligible gain. These negatives shaped the two retained mechanisms: every signal that survives is either decoder-reproducible (quantization) or training-only (pruning).

V. CONCLUSION

DCCA-GS makes two decisions inside anchor-based 3DGS compression content-aware without spending a bit of side information. A decoder-reproducible complexity multiplier, supervised in training by render sensitivity, allocates quantization precision where the image needs it. Coverage-aware fusion pruning, built on ADMM budget sparsification, decides which anchors survive by what they cover—not only by how complex they look—and removes the catastrophic low-rate failure mode at a quality cost within measurement noise. Across three large-scale UAV scenes the framework holds 58% of the HAC++ model size at matched λ within 0.27–0.61 dB, and aggregates the highest per-scene rank among 18 compared methods. Both mechanisms respect the same principle: every signal that shapes the bitstream must be recomputable from the bitstream itself.

REFERENCES

- [1] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” in *ACM SIGGRAPH*, 2023.
- [2] T. Lu, M. Yu, L. Xu, Y. Xiangli, L. Wang, D. Lin, and B. Dai, “Scaffold-gs: Structured 3d gaussians for view-adaptive rendering,” in *CVPR*, 2024.
- [3] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai, “Hac: Hash-grid assisted context for 3d gaussian splatting compression,” in *ECCV*, 2024.
- [4] —, “Hac++: Towards 100x compression of 3d gaussian splatting,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

- [5] Y. Wang, Z. Li, L. Guo, W. Yang, A. C. Kot, and B. Wen, "Contextgs: Compact 3d gaussian splatting with anchor level context model," *Advances in Neural Information Processing Systems*, vol. 37, pp. 51 532–51 551, 2024.
- [6] Y. Zhang, W. Jia, W. Niu, and M. Yin, "Gaussianspa: An optimizing-sparsifying simplification framework for compact and high-quality 3D gaussian splatting," in *CVPR*, 2025.
- [7] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang, "Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps," *Advances in Neural Information Processing Systems*, vol. 37, pp. 140 138–140 158, 2024.
- [8] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park, "Compact 3d gaussian representation for radiance field," in *CVPR*, 2024.
- [9] P. Papantonakis, G. Kopanas, B. Kerbl, A. Lanvin, and G. Drettakis, "Reducing the memory footprint of 3d gaussian splatting," *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 7, no. 1, 2024.
- [10] S. Xie, W. Zhang, C. Tang, Y. Bai, R. Lu, S. Ge, and Z. Wang, "Mesongs: Post-training compression of 3d gaussians via efficient attribute transformation," in *ECCV*, 2024.
- [11] Y. Chen, M. Li, Q. Wu, W. Lin, M. Harandi, and J. Cai, "Pcgs: Progressive compression of 3d gaussian splatting," in *AAAI*, 2026.
- [12] V. Ye, R. Li, J. Kerr, M. Turk, A. Kanazawa *et al.*, "gsplat: An open-source library for gaussian splatting," *Journal of Machine Learning Research*, 2024.
- [13] J. Balle, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," in *ICLR*, 2018.
- [14] Z. Cheng, H. Sun, M. Takeuchi, and J. Katto, "Learned image compression with discretized gaussian mixture likelihoods and attention modules," in *CVPR*, 2020.
- [15] H. Wang, H. Zhu, T. He, R. Feng, J. Deng, J. Bian, and Z. Chen, "End-to-end rate-distortion optimized 3d gaussian representation," in *ECCV*, 2024.
- [16] X. Liu, X. Wu, P. Zhang, S. Wang, Z. Li, and S. Kwong, "Compgs: Efficient 3d scene representation via compressed gaussians splatting," in *ACM Multimedia*, 2024.
- [17] Y.-T. Zhan, C.-Y. Ho, H.-B. Yang, Y.-H. Chen, J.-C. Chiang, Y.-L. Liu, and W.-H. Peng, "Cat-3dgs: A context-adaptive triplane approach to rate-distortion-optimized 3dgs compression," in *ICML*, 2025.
- [18] G. Fang and B. Wang, "Mini-splatting: Representing scenes with a constrained number of gaussians," in *ECCV*, 2024, arXiv:2403.14166.